

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members:

Rachel Luo (CruzID: 2026113)

Spring 2026

1 Introduction	2
2 Dataset	2
3 Implementation	3
3.1 Model	3
3.2 Training	5
4 Experiments	5
5 Results	6
6 Discussion	6
7 Reproducibility	8
8 Team Contributions	8
9 References	9

1 Introduction

Given a dataset of 1079 images, our task was to train a classification model that can classify 1000 images into 100 classes with above 60% accuracy. For this task, transfer learning is appropriate due to the small dataset provided. By using another model's feature extractors, all we need to focus on using the data for is classification. After selecting efficientnetv2-s as the backbone, I first experimented with different hyperparameters using K-Fold Cross Validation on small subsets, until I came up with a combination of hyperparameters that would result in a model with a validation accuracy of 90.65% and a test accuracy of 78.18% on Kaggle.

2 Dataset

Found on:

<https://www.kaggle.com/competitions/ucsc-cse-144-spring-2026-final-project/data>

The training set was organized into 100 folders labeled from 0-99, which were the classes of each image set. I loaded the images and their labels with a custom implementation of the PyTorch ImageFolder class to perform the correct class-to-index mapping. There were 100 classes and 1079 training images, stored under the directory '/train', sorted by folders named after the image classes. There were 1035 images in the test set, stored under the directory '/test'.

The training data was then split into a training set and a validation set at a ratio of 9:1 (972:107). The test set was loaded via a custom implementation of the PyTorch Dataset class, such that the image ID tags were mapped to their respective images.

These datasets were then further processed via augmentations from the Transforms v2 API:

Transforms	Parameters
Cutmix/Mixup	alpha=0.2
RandAugment	ops=2, magnitude=7, 12, 14

Both training and test inputs were processed with transforms suggested by the PyTorch documentation for EfficientnetV2-S.

Transforms	Parameters
Resize + CenterCrop	size=(384, 384), (480, 480)
Normalization	mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)

After preprocessing, all datasets were put into the PyTorch DataLoader, to be used for training, validation, and testing loops.

3 Implementation

3.1 Model

1. Pretrained backbone

I originally chose EfficientNetV2 S, since it performed better than EfficientNetV2-B0 while also having better training efficiency, and a smaller amount of total trainable parameters. I could usually terminate training at around 25 epochs with good results. My idea was because it was smaller, it could generalize better. While it did generalize better, it suffered a lot in accuracy, though, so I eventually swapped over to EfficientNetV2 L, which had more trainable parameters, but still had good training efficiency. EfficientNetV2 L worked well over a range of different input sizes, which I utilized by feeding in smaller resolution images at the beginning of training, then slowly scaled upwards, to help the model generalize better.

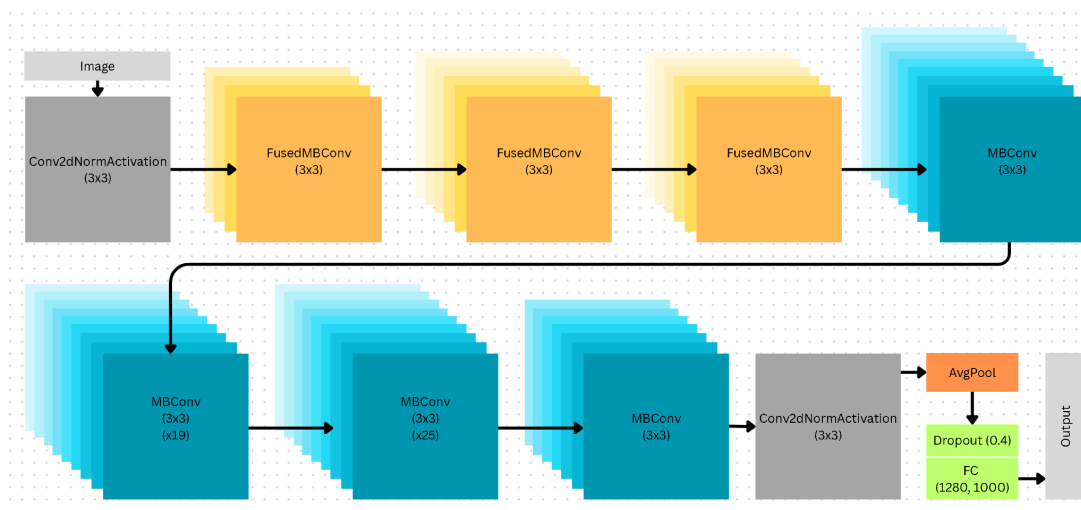


Fig 1. EfficientNetV2-L Backbone

2. Architecture changes

I changed the classifier head of the backbone model to a Dropout layer with a dropout rate of 0.5 and the last fully connected layer to have 100 outputs.

3. Fine-tuning strategy

I divided training into two phases: In phase 1, I completely unfroze the classifier head and trained that for 20 epochs, starting at a learning rate of $3e-03$.

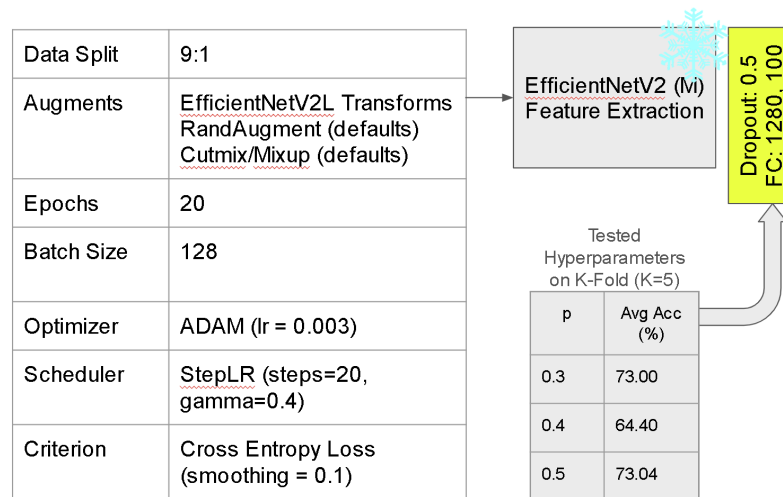


Fig 2. Phase 1 Training

Then for phase 2, taking the best model from the first phase, I unfroze one layer of the feature extractor at a time. The layer was trained with a learning rate of $1e-04$ that decayed by a factor of 0.6 for each layer unfrozen.

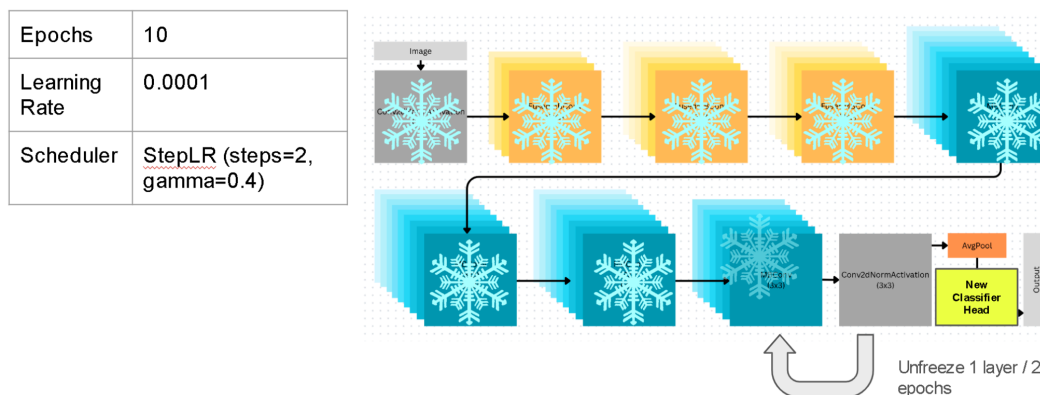


Fig 3. Phase 2 Training

3.2 Training

1. Loss function and optimizer.

Optimizer	ADAM(classifier_learning_rate=3e-03, feature_learning_rate=1e-04)
Criterion/Loss	CrossEntropyLoss(label_smoothing=0.1)
Learning Rate Scheduler	StepLR(step=5, gamma=0.3)
Batch Size	128
Epochs	Phase 1: 20, Phase 2: 10
Patience (Early Stop)	5 epochs

3. Hardware/software environment (GPU, PyTorch version).

GPU	NVIDIA T4 GPU (on Google Colab)
PyTorch version	2.11.0+cu128

4 Experiments

1. Baseline Setup (Epochs = 30)

Model	Best Training Accuracy	Best Validation Accuracy
ResNet-50	50.12	20.93
EfficientNetV2-S	94.86	46.70
VIT_b_16	1.15	0.96

Something definitely went wrong with the VIT setup, but the EfficientNetV2-S results were extremely promising, so I didn't bother looking into VIT further.

I used K-Fold Cross Validation (K=5) to test my hyperparameters, splitting my data 7:2:1 for the training set, the validation set, and then the test set. I picked the best model

based on whichever one had the best average test accuracy across all five folds. Each test was run with the best hyperparameter from the previous test.

Ablations

Hyperparameter Tested	Avg Accuracy across all folds (%)	Best Avg Test Accuracy (%)
Dropout Rate	70.14	73.04 (p=0.5)
Classifier LR	87.02	88.19 (lr=0.003)
Feature LR	89.84	90.65 (lr=0.0001)

5 Results

1. Training/validation accuracy (and loss) and any key curves/plots (optional).

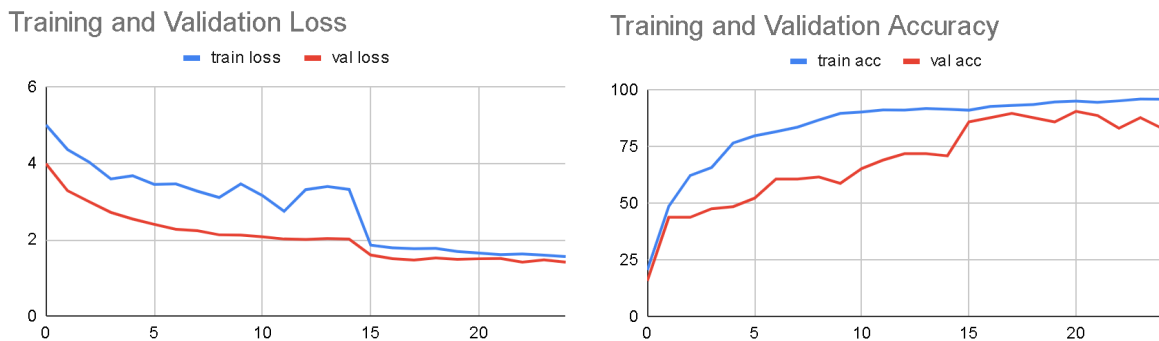


Fig 1. Loss and Accuracy Curve

The jump around epoch=15 signifies the start of Phase 2 training.

2. Kaggle public leaderboard score (and submission settings).

29

gdfsdgdghkjjk



0.79090

22

2m

6 Discussion

The hyperparameter that affected my final accuracy score the most was the dropout rate [4]. Across all my tests, a dropout rate of p=0.5 worked the best, likely due to the fact that it helped the model generalize better by reducing/penalizing larger weights.

I also played around with the split ratio of the training and validation set. Having the training data and validation data at a 9:1 split helped the model generalize better than a ratio of 8:2, since the model now had more data to train with.

RandAugment [2] was the best transformation provided by PyTorch, since it introduced a lot more randomness into the training data, making old data more “fresh” in this case. I was using individual transforms before (ColorJitter, RandRotation, RandomErasing), but I swapped to RandAugment out of convenience and better performance, since too many transformations seemed to be confusing the model too much. Another augmentation strategy I used was dynamic augmentation [6], which applied heavier augments as training went on. Passing in lower resolution features helped the model learn general patterns first, before focusing on the details.

2. Failure cases, overfitting/underfitting observations

At some point, all I got on Kaggle was 20% accuracy. This was due to the PyTorch ImageFolder, which I was using to load my training data. PyTorch sorts the folder names alphabetically, so it was converting all the folder names to strings, and sorting them that way. I fixed this by remapping all my indices separately, after classifying the test set without retraining my model, and my accuracy immediately jumped up to my best accuracy yet. Unfortunately, after that, and when I overrode the ImageFolder class to load data in better, my accuracy kept going down, so to preserve the best performing model, I stuck to remapping my class labels after the model classifies the image.

Despite that, the biggest challenge was still overfitting. At around epoch=5, the training accuracy would always jump to 90% while the validation accuracy grew at its original rate. Even after my workarounds,

I also tried implementing an STN, but it made the model too heavy, so I ended up cutting out the transformation during forward pass. I kept the layers in, though since I trained my best weights off this model, and removing it would mean I would have to go back through the model state dictionary and remove the weights assigned to those layers as well.

I also noticed that underfitting would occur after I introduced Cutmix/Mixup and other augmentations on top of RandAugment. I interpreted these results as the heavy transformations confusing the model too much. Thus, I cut out all transforms but RandAugment and randomly applied Cutmix/Mixup only 50% of the time.

3. Limitations and concrete next improvements.

A few errors that popped up frequently were “Out of Memory” errors (for STN implementation), and random disconnects from Colab during training. This slowed down my progress quite a bit, but I didn’t have a dedicated personal machine to train on. As for improvements to the model itself, I liked my classmate’s approach using Vision Language Models (VLMs). I tried implementing one, but ran out of time to complete it. For next steps, I would finish the implementation, and introduce it to my current pipeline either by ensembling it to the outcome of the EfficientnetV2-L model, or replacing the pipeline entirely.

Something else I would do to improve the model would be to add the validation set to the training set in the last few epochs of fine-tuning, just so it can learn from a few more examples to improve generalization.

One last thing I’d do, given the time, is properly retrain my model without the STN layers, just to get rid of any redundancy.

7 Reproducibility

Seed	42
Pytorch Version	2.11.0+cu128

Setup environment using `set_seed()` module from `setup_env`. Default seed is 42.

To train, open `eval.ipynb` in Google Colab and run or run on a local machine after changing necessary file paths, as directed within the `.ipynb` file.

8 Team Contributions

1. Rachel: Research, Pipeline Design, Revision, Training

9 References

TorchVision documentation / pretrained model sources.

[1]“efficientnet_v2_l — Torchvision main documentation,” Pytorch.org, 2024.
https://docs.pytorch.org/vision/main/models/generated/torchvision.models.efficientnet_v2_l.html

[2]“Transforming and augmenting images — Torchvision 0.22 documentation,” Pytorch.org, 2024. <https://docs.pytorch.org/vision/0.22/transforms.html>

[3] “Models and pre-trained weights — Torchvision main documentation,” Pytorch.org, 2024. <https://docs.pytorch.org/vision/main/models.html>

Other papers or tutorials used.

[4] J. Shimazoe, K. Arai, and M. Oda, “Method for Hyperparameter Tuning of EfficientNetV2-based Image Classification by Deliberately Modifying Optuna Tuned Result,” *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 12, 2023, doi: <https://doi.org/10.14569/ijacsa.2023.014124>.

[5] Tan, M., & Le, Q. (2021, July). Efficientnetv2: Smaller models and faster training. In International conference on machine learning (pp. 10096-10106). PMLR.

[6] X. Yu, H. Zhao, M. Zhang, Y. Wei, L. Zhou, and L. Ou, “DynamicAug: Enhancing Transfer Learning Through Dynamic Data Augmentation Strategies Based on Model State,” *Neural Processing Letters*, vol. 56, no. 3, May 2024, doi: <https://doi.org/10.1007/s11063-024-11626-9>.